

Regular Expressions

A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.

RegEx can be used to check if a string contains the specified search pattern.

RegEx Module

Python has a built-in module called re, which can be used to work with Regular Expressions.

Import the re module:

```
In [1]: import re #module
```

```
In [ ]:
```

```
In [2]: dir(re)
```

```
Out[2]: ['A',
         'ASCII',
         'DEBUG',
         'DOTALL',
         'I',
         'IGNORECASE',
         'L',
         'LOCALE',
         'M',
         'MULTILINE',
         'Match',
         'Pattern',
         'RegexFlag',
         'S',
         'Scanner',
         'T',
         'TEMPLATE',
         'U',
         'UNICODE',
         'VERBOSE',
         'X',
         '_MAXCACHE',
         '__all__',
         '__builtins__',
         '__cached__',
         '__doc__',
         '__file__',
         '__loader__',
         '__name__',
         '__package__',
         '__spec__',
         '__version__',
         '_cache',
         '_compile',
         '_compile_repl',
         '_expand',
         '_locale',
         '_pickle',
         '_special_chars_map',
         '_subx',
         'compile',
         'copyreg',
         'enum',
         'error',
         'escape',
         'findall',
         'finditer',
         'fullmatch',
         'functools',
         'match',
         'purge',
         'search',
         'split',
         'sre_compile',
         'sre_parse',
         'sub',
```

```
'subn',  
'template']
```

Function	Description
<u>findall</u>	Returns a list containing all matches
<u>search</u>	Returns a <u>Match object</u> if there is a match anywhere in the string
<u>split</u>	Returns a list where the string has been split at each match
<u>sub</u>	Replaces one or many matches with a string

1.findall

```
In [15]: x = " what do you do do "  
y = re.findall("do",x)  #function(attributes)  
print(y)  
len(y)  
y.count("do")  
  
['do', 'do', 'do']
```

Out[15]: 3

```
In [2]: x="do what do you do"  
re.findall("do",x)
```

Out[2]: ['do', 'do', 'do']

```
In [6]: import re  
x = "The rain "  
y = re.findall("sun", x)  
print(y)
```

[]

search

```
In [20]: text = "wel come to ooty"  
y=re.search("\s",text)  
y
```

Out[20]: <re.Match object; span=(3, 4), match=' '>

```
In [28]: #w=0
#e=1
#l=2
#" "=3
for i in range(3,4):
    print(i)
```

3

re.search() indeed returns only the first occurrence of the matched pattern.

```
In [24]: text = "wel come to ooty"
y=re.search("\s",text)
print(y.start())
print(y.end())
```

3

4

```
In [13]: import re

txt = "The rain in Spain" #3,4
x = re.search("\s", txt)
print(x.end())
print("The first white-space character is located in position:", x.start())
```

4

The first white-space character is located in position: 3

```
In [25]: text = "welcome hai" #c-3,o-4
y=re.search("co",text)
print(y)
print(y.start())
print(y.end())
```

<re.Match object; span=(3, 5), match='co'>

3

5

```
In [17]: for i in range(3,5):
    print(i)
```

3

4

```
In [18]: txt = "The rain in Spain"
x = re.search("ai", txt)# 5,6
print(x)
#t=0, h=1 , e=2 , " "=3, r=4, "a "=5, "i "=6
```

<re.Match object; span=(5, 7), match='ai'>

```
In [23]: txt = "The rain in Spain"
x = re.search("a", txt)
print(x)

<re.Match object; span=(5, 6), match='a'>
```

```
In [30]: txt = "The rain in Spain"
x = re.findall("\s",txt)
print(x)
len(x)

[' ', ' ', ' ', ' ']
```

Out[30]: 3

split

```
In [31]: txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)

['The', 'rain', 'in', 'Spain']
```

```
In [23]: txt = "The rain in Spain"
x = re.split("ai", txt)
print(x)

['The r', 'n in Sp', 'n']
```

```
In [32]: import re

txt = "The rain in Spain"
x = re.split("\s", txt,2) #You can control the number of occurrences by specifying
print(x)

['The', 'rain', 'in Spain']
```

sub

```
In [33]: txt = "The rain in Spain"
x = re.sub("\s", "9", txt)
print(x)

The9rain9in9Spain
```

```
In [34]: txt = "The rain in Spain"
x = re.sub("a", "A", txt)
print(x)
```

The rAin in SpAin

```
In [35]: txt = "The rain in Spain any"
x = re.sub("a", "$", txt,2)
print(x)
```

The r\$in in Sp\$in any

re.match():

Functionality: Checks for a match only at the beginning of the string.

```
In [36]: import re

# Test string
text = 'Then quick brown fox' #t-0,h-1,e-2

match = re.match("The", text)

match
```

```
Out[36]: <re.Match object; span=(0, 3), match='The'>
```

It returns a match object if the pattern matches at the beginning of the string, or None if there is no match.

```
In [39]: import re

# Test string
text = 'Then quick brown fox'

match = re.match("fox", text)

match
```

revisit

```
In [40]: import re

# Define a pattern
pattern = 'fox'

# Test string
text = 'The quick brown fox' #f-16,o-17,x-18

match = re.search(pattern, text) #checks the entire, first match

match
```

Out[40]: <re.Match object; span=(16, 19), match='fox'>

```
In [42]: import re

# Define a pattern
pattern = 'fox'

# Test string
text = 'The quick brown fox fox' #f-16,o-17,x-18

match = re.findall(pattern, text)

len(match)
```

Out[42]: 2

```
In [ ]: # modules
```

```
In [44]: import random as rd
```



```
In [45]: dir(rd)
```

```
Out[45]: ['BPF',
          'LOG4',
          'NV_MAGICCONST',
          'RECIP_BPF',
          'Random',
          'SG_MAGICCONST',
          'SystemRandom',
          'TWOPI',
          '_Sequence',
          '_Set',
          '__all__',
          '__builtins__',
          '__cached__',
          '__doc__',
          '__file__',
          '__loader__',
          '__name__',
          '__package__',
          '__spec__',
          '_accumulate',
          '_acos',
          '_bisect',
          '_ceil',
          '_cos',
          '_e',
          '_exp',
          '_floor',
          '_inst',
          '_log',
          '_os',
          '_pi',
          '_random',
          '_repeat',
          '_sha512',
          '_sin',
          '_sqrt',
          '_test',
          '_test_generator',
          '_urandom',
          '_warn',
          'betavariate',
          'choice',
          'choices',
          'expovariate',
          'gammavariate',
          'gauss',
          'getrandbits',
          'getstate',
          'lognormvariate',
          'normalvariate',
          'paretovariate',
          'randbytes',
          'randint',
          'random',
          'randrange',
          'sample',
          'seed',
```

```
'setstate',  
'shuffle',  
'triangular',  
'uniform',  
'vonmisesvariate',  
'weibullvariate']
```

```
In [53]: rd.random()
```

```
Out[53]: 0.4074133641151003
```

```
In [58]: rd.randint(1,10)
```

```
Out[58]: 9
```

```
In [71]: rd.randrange(2,1000,2)    #2,4,6,8,10...1000
```

```
Out[71]: 250
```

```
In [80]: x=[3,8,0,5,4]
```

```
In [81]: rd.choice(x)
```

```
Out[81]: 4
```

```
In [82]: rd.shuffle(x)
```

```
In [83]: x
```

```
Out[83]: [5, 4, 0, 3, 8]
```

```
In [85]: rd.sample(x,3)
```

```
Out[85]: [8, 5, 4]
```

```
In [88]: import statistics as st
```

```
In [89]: dir(st)
```

```
Out[89]: ['Counter',
          'Decimal',
          'Fraction',
          'NormalDist',
          'StatisticsError',
          '__all__',
          '__builtins__',
          '__cached__',
          '__doc__',
          '__file__',
          '__loader__',
          '__name__',
          '__package__',
          '__spec__',
          '_coerce',
          '_convert',
          '_exact_ratio',
          '_fail_neg',
          '_find_lteq',
          '_find_rteq',
          '_isfinite',
          '_normal_dist_inv_cdf',
          '_ss',
          '_sum',
          'bisect_left',
          'bisect_right',
          'erf',
          'exp',
          'fabs',
          'fmean',
          'fsum',
          'geometric_mean',
          'groupby',
          'harmonic_mean',
          'hypot',
          'itemgetter',
          'log',
          'math',
          'mean',
          'median',
          'median_grouped',
          'median_high',
          'median_low',
          'mode',
          'multimode',
          'numbers',
          'pstdev',
          'pvariance',
          'quantiles',
          'random',
          'sqrt',
          'stdev',
          'tau',
          'variance']
```

In [90]: `1000/1200`

Out[90]: 0.8333333333333334

In [91]: `120+100+150`

Out[91]: 370

In [92]: `370/3`

Out[92]: 123.33333333333333

In [93]: `x=[120,100,150]`
`st.mean(x)`

Out[93]: 123.33333333333333

In [94]: `st.median(x)`

Out[94]: 120

In [95]: `st.stdev(x)`

Out[95]: 25.16611478423583

In []: